

Topic I: Heaps as Priority Queues — Foundations & Operations

Priority Queue ADT, Decrease-Key Mechanics, Extract-Min, Array Bounds

Part 1: Canonical Benchmark PYQs (Questions 1 to 3)

Q1. Priority Queue Operations Time Bounds

GATE CS 2003 / DA Benchmark | MCQ

In a binary min-heap implementing a Priority Queue with n elements, what are the worst-case asymptotic complexities of *Insert*, *Extract-Min*, and *Find-Min*?

- (A) $O(\log n)$, $O(\log n)$, $O(1)$ (B) $O(1)$, $O(\log n)$, $O(1)$
 (C) $O(\log n)$, $O(1)$, $O(\log n)$ (D) $O(n)$, $O(\log n)$, $O(1)$

Answer: (A) Step-by-step Solution: In a binary min-heap: (1) *Find-Min* takes $O(1)$ time because the minimum key is stored at the root array index 0 (or 1). (2) *Insert* places the node at the next open leaf and bubbles up along a path of length at most $\lfloor \log_2 n \rfloor$, costing $O(\log n)$. (3) *Extract-Min* replaces the root with the last leaf and sifts down along the tree height, taking $O(\log n)$.

Q2. Decrease-Key Operation Mechanics

GATE CS 2014 | MCQ

Suppose the priority of an arbitrary element located at known array index i in an n -element binary min-heap is decreased. The procedure executed to restore the min-heap invariant is:

- (A) Bubble-down (Max/Min-Heapify downward) in $O(\log n)$ (B) Bubble-up (sift-up toward the root) in $O(\log n)$
 (C) Rebuild the entire heap using *Build-Heap* in $O(n)$ (D) Constant time swap with the root in $O(1)$

Answer: (B) Step-by-step Solution: Decreasing a key in a min-heap makes it strictly smaller than before. It cannot violate the min-heap condition with respect to its descendants (which are already greater than or equal to its old larger value). However, it may now be smaller than its parent. Hence, it must percolate upward (bubble-up) towards the root by swapping with its parent until the heap property is restored. Worst-case path length is $O(\log n)$.

Q3. Finding Maximum Element in a Min-Heap

GATE CS 2011 / DA Sample | MCQ

What is the worst-case time complexity to search for the **maximum** element in a binary min-heap containing n elements?

- (A) $\Theta(1)$ (B) $\Theta(\log n)$
 (C) $\Theta(n)$ (D) $\Theta(n \log n)$

Answer: (C) Step-by-step Solution: In a min-heap, internal nodes are smaller than their children. Therefore, the maximum key cannot reside at any internal node; it must be located at one of the leaf nodes. A binary heap contains $\lceil n/2 \rceil$ leaves. Because leaves are unsorted relative to each other, finding the maximum requires a linear scan over all $\lceil n/2 \rceil$ leaves, taking $\Theta(n)$ time.

Q4. Merging k Sorted Arrays Each of Size n

GATE CS 2007 / DA Benchmark | MCQ

What is the most optimal time complexity to merge k sorted arrays, each containing n elements, into a single sorted output array of size $k \cdot n$?

- (A) $\Theta(k \cdot n \log k)$ (B) $\Theta(k \cdot n \log n)$
 (C) $\Theta(k^2 \cdot n)$ (D) $\Theta(n \log k)$

Answer: (A) Step-by-step Solution: Initialize a binary min-heap with the first element of each of the k sorted arrays in $O(k)$ time. Then, repeatedly extract the root (minimum element) and insert the next available element from the array that provided the extracted root. There are $k \cdot n$ total elements, and each extraction/insertion step takes $O(\log k)$. Total runtime is $\Theta(k \cdot n \log k)$.

Q5. Extracting k -th Largest Element from an Array

GATE CS 2015 | MCQ

Given an unsorted array of N elements where N is very large and $k \ll N$, the most space-efficient method to determine the k largest elements in $O(N \log k)$ time uses:

- (A) A Min-Heap of size k (B) A Max-Heap of size N
 (C) A Max-Heap of size k (D) A Min-Heap of size N

Answer: (A) Step-by-step Solution: Build a Min-Heap of the first k elements in $O(k)$. For each remaining element x , compare with the heap root (the smallest among the current top k). If $x > \text{root}$, replace root and heapify in $O(\log k)$. Space complexity is strictly $O(k)$ and total time is $O(N \log k)$. A Max-Heap of size k cannot determine whether a new element belongs without checking all internal nodes.

Q6. Extracting k Smallest from a Max-Heap of Size n

GATE CS 2006 | MCQ

If an input dataset is already stored in a binary **max-heap** of size n , what is the tightest time bound to output the k smallest elements in sorted order?

- (A) $O(n)$ for any k (B) $O(k \log n)$
 (C) $O(n + k \log k)$ (D) $O(k)$

Answer: (C) Step-by-step Solution: A max-heap gives no direct structural speedup for locating minimum elements; all minimum elements reside among the $\lceil n/2 \rceil$ leaves. Selecting the k smallest elements from $n/2$ leaves requires either running QuickSelect in $O(n)$ followed by sorting them in $O(k \log k)$, yielding $O(n + k \log k)$.

Q7. Negation of Nested Quantifiers

GATE DA 2024 / CS 2013 | MCQ

Which of the following first-order logic formulas is logically equivalent to the negation of the sentence: $\forall x \exists y (P(x, y) \rightarrow Q(x, y))$?

- (A) $\exists x \forall y (P(x, y) \wedge \neg Q(x, y))$ (B) $\exists x \forall y (\neg P(x, y) \vee Q(x, y))$
 (C) $\forall x \exists y (\neg P(x, y) \wedge Q(x, y))$ (D) $\exists x \exists y (P(x, y) \wedge \neg Q(x, y))$

Answer: (A) Step-by-step Solution: $\neg[\forall x \exists y (P(x, y) \rightarrow Q(x, y))] \equiv \exists x \neg[\exists y (P(x, y) \rightarrow Q(x, y))] \equiv \exists x \forall y \neg[P(x, y) \rightarrow Q(x, y)]$. Applying De Morgan's law yields: $\exists x \forall y (P(x, y) \wedge \neg Q(x, y))$.

Q8. Universal Quantifier Distribution over Connectives

GATE CS 2008 | MSQ

Which of the following logical equivalences hold strictly for all predicates $P(x)$ and $Q(x)$ over a non-empty domain? [Select all that apply]

- (A) $\forall x (P(x) \wedge Q(x)) \equiv (\forall x P(x)) \wedge (\forall x Q(x))$ (B) $\exists x (P(x) \vee Q(x)) \equiv (\exists x P(x)) \vee (\exists x Q(x))$
 (C) $\forall x (P(x) \vee Q(x)) \equiv (\forall x P(x)) \vee (\forall x Q(x))$ (D) $(\forall x P(x)) \rightarrow (\exists x P(x))$

Answer: (A), (B), and (D) Step-by-step Solution: Universal quantifiers distribute over conjunction $\wedge$ (A), and existential quantifiers distribute over disjunction $\vee$ (B). In non-empty domains, if every element satisfies P , then there exists at least one element satisfying P (D). (C) is invalid: let domain be integers, $P(x)$: "x is even", $Q(x)$: "x is odd"; every integer is even or odd (LHS is True), but not all are even nor all are odd (RHS is False).

Q9. Scope of Quantifiers and Variable Independence

GATE CS 2016 | MCQ

Let formula α be: $(\forall x P(x)) \rightarrow Q$, where Q contains no occurrences of variable x . Formula α is logically equivalent to:

- (A) $\exists x (P(x) \rightarrow Q)$ (B) $\forall x (P(x) \rightarrow Q)$
 (C) $\neg \exists x (P(x) \wedge Q)$ (D) $(\forall x \neg P(x)) \vee Q$

Answer: (A) Step-by-step Solution: $(\forall x P(x)) \rightarrow Q \equiv \neg(\forall x P(x)) \vee Q \equiv (\exists x \neg P(x)) \vee Q$. Because x does not appear free in Q , the existential quantifier can be pulled over the disjunction: $\exists x (\neg P(x) \vee Q) \equiv \exists x (P(x) \rightarrow Q)$.

Q10. Canonical Translation of Conditional Generalizations

GATE CS 2014 / DA Standard | MCQ

Consider the English statement: "Every student who studies hard passes the examination."

Predicates: $S(x)$: "x is a student", $H(x)$: "x studies hard", $P(x)$: "x passes the examination". The correct representation in First-Order Logic is:

- (A) $\forall x ((S(x) \wedge H(x)) \rightarrow P(x))$ (B) $\forall x (S(x) \rightarrow (H(x) \wedge P(x)))$
 (C) $\exists x (S(x) \wedge H(x) \wedge P(x))$ (D) $\forall x (S(x) \wedge H(x) \wedge P(x))$

Answer: (A) Step-by-step Solution: "Every student who studies hard" restricts the condition to entities satisfying both $S(x)$ and $H(x)$. The property asserted of all such individuals is $P(x)$. Universal quantification over a conditional property requires the implication arrow $\rightarrow$. Formula: $\forall x ((S(x) \wedge H(x)) \rightarrow P(x))$. (Using $\wedge$ instead of $\rightarrow$ as in (D) asserts that *all objects in the universe* are hard-studying students who pass).

Q11. Merging k Sorted Lists Total Comparisons Count

New Practice | NAT

We are merging $k = 4$ sorted arrays, each containing 3 elements, into a single sorted list using a min-heap of size 4. What is the total number of element extractions performed from the min-heap?

Answer: 12 Step-by-step Solution: The final merged sorted array contains $k \times n = 4 \times 3 = 12$ total items. In the standard k -way merge algorithm, every element placed into the merged output array corresponds to exactly one *Extract-Min* operation from the active min-heap. Hence, exactly 12 extractions are executed.

Q12. Translation of "Some Students Like Every Teacher"

New Practice | MCQ

Let $S(x)$: "x is a student", $T(y)$: "y is a teacher", and $L(x, y)$: "x likes y". Which formula correctly expresses: "There is some student who likes every teacher"?

- (A) $\exists x (S(x) \wedge \forall y (T(y) \rightarrow L(x, y)))$ (B) $\forall y (T(y) \rightarrow \exists x (S(x) \wedge L(x, y)))$
 (C) $\exists x \forall y (S(x) \wedge T(y) \wedge L(x, y))$ (D) $\forall x (S(x) \rightarrow \forall y (T(y) \rightarrow L(x, y)))$

Answer: (A) Step-by-step Solution: "There is some student" requires an outer existential quantifier $\exists x$ with conjunction $S(x) \wedge \dots$. For that student, the statement holds for "every teacher", which requires universal quantification $\forall y$ conditioned on y being a teacher: $T(y) \rightarrow L(x, y)$. Putting them together gives $\exists x (S(x) \wedge \forall y (T(y) \rightarrow L(x, y)))$. Notice (B) expresses "Every teacher is liked by some student" (allowing different students for different teachers).

Practice Set: Priority Queue Algorithms & Operations (Practice 13 to 15)

Q13. Index Calculations in 0-Based Heap Representation

New Practice | NAT

In a priority queue array using 0-based indexing, an element resides at index 14. What is the array index of its parent node?

Answer: 6 Step-by-step Solution: In a 0-based binary heap, for any child at index $i > 0$, the parent is located at index $\lfloor (i - 1) / 2 \rfloor$. For index $i = 14$: $\lfloor (14 - 1) / 2 \rfloor = \lfloor 13 / 2 \rfloor = 6$. (Verification: children of parent 6 are $2(6) + 1 = 13$ and $2(6) + 2 = 14$).

Q14. Priority Queue Implementation via Unsorted Array vs Binary Heap

New Practice | MSQ

Which of the following comparisons between an *unsorted array* and a *binary min-heap* implementation of a Priority Queue is/are **CORRECT**? [Select all that apply]

- | | |
|---|--|
| (A) Unsorted array performs <i>Insert</i> in $O(1)$ worst-case time. | (B) Unsorted array performs <i>Extract-Min</i> in $O(n)$ worst-case time. |
| (C) Binary min-heap achieves strictly better amortized time for both operations combined. | (D) Unsorted array can locate the minimum element in $O(\log n)$ time using binary search. |

Answer: (A), (B), and (C) Step-by-step Solution: (A) Appending to an unsorted array takes $O(1)$. (B) Extract-min on an unsorted array requires scanning all n elements to find the minimum, costing $O(n)$. (C) In applications with mixed inserts and extracts, binary heap offers $O(\log n)$ per operation, outperforming $O(n)$ per extract. (D) is false: binary search requires the array to be sorted.

Q15. Running Median Using Dual Priority Queues

New Practice | MCQ

To maintain the continuous median of a numerical stream of N numbers with $O(\log N)$ insertion time and $O(1)$ median query time, the standard optimal approach uses:

- | | |
|--|---|
| (A) A Max-Heap for the smaller half and a Min-Heap for the larger half | (B) A single Max-Heap of size N |
| (C) Two Min-Heaps storing odd and even entries respectively | (D) A single Min-Heap updated using QuickSelect |

Answer: (A) Step-by-step Solution: By dividing elements into two halves, a Max-Heap stores the smaller half (with the maximum of the small half at its root) and a Min-Heap stores the larger half (with the minimum of the large half at its root). Balanced sizes ensure the median is available directly at one or both roots in $O(1)$ time, with $O(\log N)$ rebalancing per insertion.

Q16. English Statement Translation Suite: Statement 1

New Practice | MCQ

Translate the statement: "No computer scientist is unaware of all programming languages."

Predicates: $CS(x)$: "x is a computer scientist", $PL(y)$: "y is a programming language", $A(x, y)$: "x is aware of y".

- (A) $\neg \exists x (CS(x) \wedge \forall y (PL(y) \rightarrow \neg A(x, y)))$ (B) $\forall x (CS(x) \rightarrow \forall y (PL(y) \rightarrow \neg A(x, y)))$
 (C) $\exists x (CS(x) \wedge \exists y (PL(y) \wedge A(x, y)))$ (D) $\neg \forall x (CS(x) \rightarrow \exists y (PL(y) \wedge A(x, y)))$

Answer: (A) Step-by-step Solution: "No computer scientist has property P" translates to $\neg \exists x (CS(x) \wedge P(x))$. Here property $P(x)$ is "x is unaware of all programming languages", which is $\forall y (PL(y) \rightarrow \neg A(x, y))$. Combining these yields: $\neg \exists x (CS(x) \wedge \forall y (PL(y) \rightarrow \neg A(x, y)))$, which is equivalent to saying every CS person is aware of at least one PL: $\forall x (CS(x) \rightarrow \exists y (PL(y) \wedge A(x, y)))$.

Q17. English Statement Translation Suite: Statement 2

New Practice | MCQ

Translate the statement: "Every person has someone who loves them, but not everyone loves someone."

Predicates: $P(x)$: "x is a person", $L(x, y)$: "x loves y".

- (A) $[\forall x (P(x) \rightarrow \exists y (P(y) \wedge L(y, x)))] \wedge \neg [\forall x (P(x) \rightarrow \exists y (P(y) \wedge L(x, y)))]$ (B) $[\forall x (P(x) \rightarrow \exists y (P(y) \wedge L(x, y)))] \wedge \neg [\forall x (P(x) \rightarrow \exists y (P(y) \wedge L(y, x)))]$
 (C) $\forall x \exists y (P(x) \wedge P(y) \wedge L(y, x) \wedge \neg L(x, y))$ (D) $[\forall x \forall y (L(y, x))] \wedge [\exists x \forall y (\neg L(x, y))]$

Answer: (A) Step-by-step Solution: The first clause is: "For every person x, there exists some person y who loves x" $\rightarrow \forall x (P(x) \rightarrow \exists y (P(y) \wedge L(y, x)))$. The second clause is: "It is not the case that every person x loves some person y" $\rightarrow \neg \forall x (P(x) \rightarrow \exists y (P(y) \wedge L(x, y)))$. The conjunction of both is option (A).

Q18. English Statement Translation Suite: Statement 3

New Practice | MCQ

Translate the statement: "There is a student who solved more than 5 problems and received an A grade."

Predicates: $S(x)$: "x is a student", $M(x)$: "x solved > 5 problems", $G(x)$: "x received an A grade".

- (A) $\exists x (S(x) \wedge M(x) \wedge G(x))$ (B) $\exists x (S(x) \rightarrow (M(x) \wedge G(x)))$
 (C) $\forall x ((S(x) \wedge M(x)) \rightarrow G(x))$ (D) $\exists x (S(x) \wedge (M(x) \rightarrow G(x)))$

Answer: (A) Step-by-step Solution: The statement asserts the existence of an entity that simultaneously satisfies three predicates: being a student, solving > 5 problems, and receiving an A grade. The standard FOL representation of existential property conjunction is $\exists x (S(x) \wedge M(x) \wedge G(x))$. (B) is a common pitfall: an implication under $\exists$ is vacuously satisfied by any entity that is not a student.

Q19. Quantifier Alternation Non-Equivalence

New Practice | MCQ

Which of the following implications between alternating quantifiers is **VALID** (holds in all domains) but whose converse is **INVALID**?

- (A) $(\exists x \forall y P(x, y)) \rightarrow (\forall y \exists x P(x, y))$ (B) $(\forall y \exists x P(x, y)) \rightarrow (\exists x \forall y P(x, y))$
 (C) $(\forall x \forall y P(x, y)) \rightarrow (\exists x \exists y \neg P(x, y))$ (D) $(\exists x \exists y P(x, y)) \rightarrow (\forall x \forall y P(x, y))$

Answer: (A) Step-by-step Solution: If there exists a single entity x that is related to every y (LHS), then for every y , there certainly exists some x related to it (RHS). The converse is invalid: "Everyone has a mother" $(\forall y \exists x M(x, y))$ is true, but "There is someone who is the mother of everyone" $(\exists x \forall y M(x, y))$ is false.

Q20. Skolem Constant vs Skolem Function

New Practice | MSQ

When converting the first-order sentence $\forall x \exists y (P(x) \rightarrow Q(x, y))$ into standard Conjunctive Normal Form (CNF) via Skolemization: [Select all that apply]

- (A) Variable y is replaced by a Skolem function $f(x)$ dependent on x . (B) Variable y can be replaced by a Skolem constant c .
 (C) The universal quantifier $\forall x$ can be dropped with variables implicitly universally quantified. (D) The resulting clause is $\neg P(x) \vee Q(x, f(x))$.

Answer: (A), (C), and (D) Step-by-step Solution: Because $\exists y$ falls within the scope of universal quantifier $\forall x$, the value of y may depend on x . Hence, y must be replaced by a Skolem function $f(x)$, not a constant (ruling out B). Rewriting the implication: $\neg P(x) \vee Q(x, f(x))$, with universal quantifiers dropped implicitly.

Q21. Domain Modeling: Models Satisfying Uniqueness Condition

New Practice | NAT

Let domain $D = \{1, 2\}$. We define the sentence: $\exists x \forall y (R(x, y))$. A relation R on $D \times D$ has $2^4 = 16$ possible interpretations. How many of these 16 interpretations make the sentence **TRUE**?

Answer: 7 Step-by-step Solution: The sentence asserts that either row 1 of R is all True, or row 2 of R is all True (or both). Let A be the event that $R(1, 1) = R(1, 2) = 1$. Since row 2 has 2 entries (4 choices), $|A| = 2^2 = 4$. Let B be the event that row 2 is all True; $|B| = 4$. The intersection $A \cap B$ is the single interpretation where all 4 entries are True; $|A \cap B| = 1$. By inclusion-exclusion: $|A \cup B| = 4 + 4 - 1 = 7$.

Q22. Merging k Sorted Lists Auxiliary Memory Bound

New Practice | MCQ

When merging k sorted arrays of total combined length N using an active min-heap, what is the exact auxiliary space required by the priority queue data structure alone?

- (A) $O(k)$ (B) $O(N)$
(C) $O(N \log k)$ (D) $O(k^2)$

Answer: (A) Step-by-step Solution: At any point during execution, the min-heap contains at most one candidate element from each of the k arrays. Thus, the heap size is capped at k , requiring strictly $O(k)$ auxiliary memory regardless of total elements N .

Q23. Vacuous Truth in FOL Conditional Statements

New Practice | MSQ

Consider the sentence: $\forall x (P(x) \rightarrow Q(x))$ evaluated in a non-empty domain D . Under which conditions is this sentence **TRUE**? [Select all that apply]

- (A) No element in domain D satisfies $P(x)$. (B) Every element in domain D satisfies $Q(x)$.
(C) Every element that satisfies $P(x)$ also satisfies $Q(x)$. (D) At least one element in domain D satisfies both $P(x)$ and $Q(x)$.

Answer: (A), (B), and (C) Step-by-step Solution: (A) If $P(x)$ is False for all elements, the implication $False \rightarrow \dots$ evaluates to True for all x (vacuous truth). (B) If $Q(x)$ is True for all elements, $\dots \rightarrow True$ is always True. (C) is the definition of standard satisfaction. (D) is insufficient because another element might make P True and Q False, violating the universal quantification.

Q24. Heap Sift-Down Worst-Case Comparison Count

New Practice | NAT

In a binary min-heap with 7 elements (a complete binary tree of height 2), an *Extract-Min* operation is executed. What is the maximum number of element-to-element comparisons performed during the single sift-down phase?

Answer: 4 Step-by-step Solution: The replacement root at height 2 must sift down to a leaf at height 0 (traversing 2 levels). At each level: (1) 1 comparison between the two children to find the smaller child; (2) 1 comparison between the current node and the smaller child. That is 2 comparisons per level. Over 2 levels of downward descent: $2 \times 2 = 4$ total comparisons.